Grammatical Error Correction on C4_200M

Bag-of-Words → LSTM → Attention → Transformer

Abdullah Ahmed (202200206) Ibrahim Hanafy (202200518)

CIE 555 — Neural Networks and Deep Learning

Instructor: Dr. Ibrahim Swelam TAs: Aya Abdelaziz, Mahmoud Farahat

May 2026

The Problem: Grammatical Error Correction

Rewrite an ungrammatical sentence into its corrected form.

Example

Input (corrupted): *"She go to school yesterday and learning many thing."*

Output (corrected): *"She went to school yesterday and learned many things."*

- Errors: verb tense, agreement, articles, plurals, punctuation, spelling.
- The output is a **whole new sentence** — a sequence-to-sequence task.

The Data: C4_200M Subset

C4_200M — clean web text automatically corrupted into `(corrupted, clean)` pairs.

Split	Pairs
Train	850,000
Validation	50,000
Test	100,000
Total	1,000,000

- 16k Byte-Level BPE tokenizer.
- Max length 64 tokens.
- Same splits + tokenizer for every model (no leakage).

Four Models of Increasing Power

#	Model	What it adds
1	Bag-of-Words (TF–IDF)	Baseline — only *detects* errors
2	LSTM encoder–decoder	First corrector; fixed-size bottleneck
3	LSTM + Bahdanau attn	Decoder re-queries the full source
4	Transformer	Self-attention; no recurrence

Model 1 *detects*. Models 2–4 *correct*.

Model 1 — Bag-of-Words Baseline

- Each pair $\rightarrow$ two samples: corrupted = ungrammatical (0), clean = grammatical (1).
- Features: TF–IDF, unigrams + bigrams, 50k vocab (fit on train only).
- Classifiers: Logistic Regression & Linear SVM.

Result

$\sim$**62 %** detection accuracy — barely above 50 % chance.
Surface word statistics cannot solve GEC. $\rightarrow$ need seq2seq.

Model 2 — LSTM Encoder–Decoder

- BiLSTM encoder compresses input into **one** 512-dim state.
- LSTM decoder generates the correction token by token.
- Teacher forcing $1.0 \rightarrow 0.5$; 19.55 M params; 3 epochs.

The bottleneck

After the first step the decoder never "sees" the source again — everything is squeezed into 512 numbers.

Test GLEU **0.213** (greedy) — weakest corrector.

Model 3 — LSTM + Bahdanau Attention

Keep *all* encoder states; the decoder attends to them every step.

$$\text{score}(s_t, h_i) = v^\top \tanh(W_s s_t + W_h h_i), \quad \alpha = \mathrm{softmax}(\text{score}), \quad c_t = \textstyle\sum_i \alpha_{t,i} h_i$$

- Removes the bottleneck; alignment weights are interpretable.
- Padding masked with $-\infty$ before softmax.
- 29.06 M params.

Result — the decisive jump

GLEU **0.213 → 0.599** $(+0.386)$. Attention matters most.

Model 4 — Transformer

- No recurrence — self-attention only.
- $d_{\text{model}}=256$, 8 heads, 3+3 layers, FFN 512.
- Sinusoidal positional encoding; causal mask in decoder.
- Label smoothing 0.1; warmup LR; weight tying.

Smallest & strongest

Only **8.05 M** params (3.6$\times$ fewer than Model 3).
Best validation GLEU **0.613**.

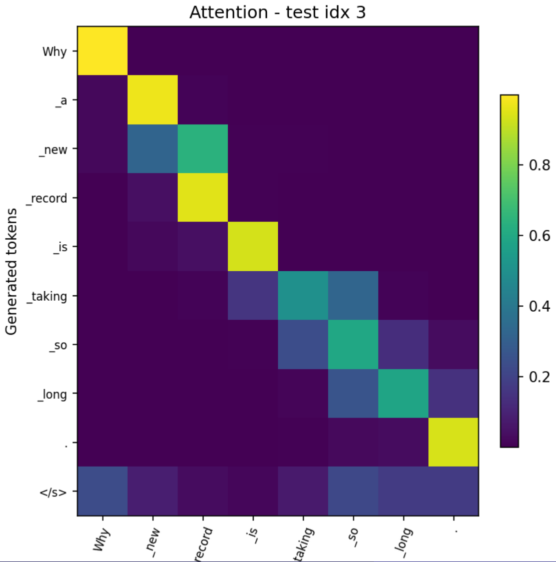
Self-attention: every token directly attends to every other — $O(1)$ path length, fully parallel.

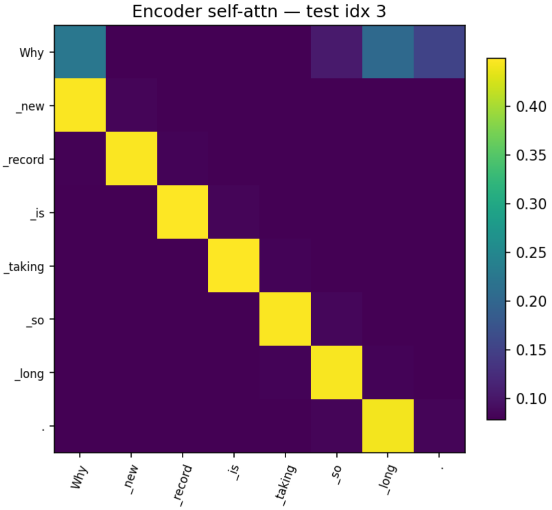
Comparative Results — Test Set

Model	Decoding	Params	EM	GLEU
LSTM (no attention)	greedy	19.55 M	$\approx$0.008	0.213
LSTM (no attention)	beam-4	19.55 M	–	0.256
LSTM + Bahdanau	greedy	29.06 M	0.024	0.599
LSTM + Bahdanau	beam-4	29.06 M	0.026	**0.618**
Transformer	greedy	8.05 M	0.022	**0.618**
Transformer	beam-4	8.05 M	0.020	**0.618**

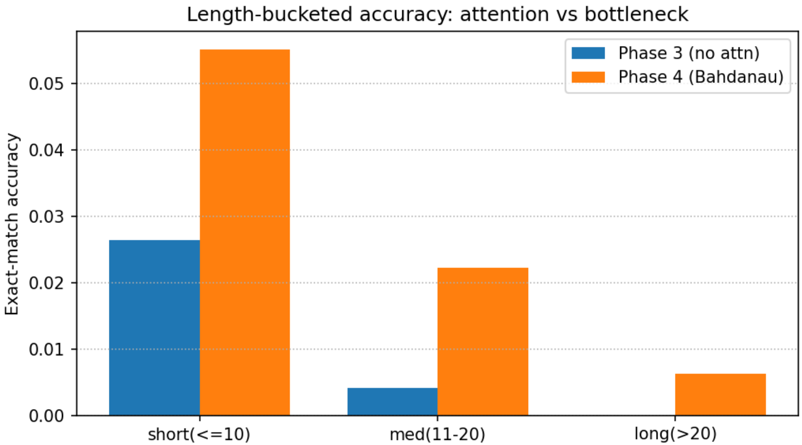
- Beam search helps the LSTM, not the (well-calibrated) Transformer.
- Transformer ties Model 3 on GLEU with 3.6$\times$ fewer parameters.

Attention Is Interpretable





Behaviour by Sentence Length



All models do best on short sentences; the bottleneck LSTM degrades fastest as sentences get longer.

Grammar-Rule Analysis of Outputs

Learned well (local syntax):

- Subject–verb agreement: *"What are this job"* $\rightarrow$ *"What is this job"*
- Articles: *"Why new record"* $\rightarrow$ *"Why the new record"*
- Verb tense, punctuation, spacing.

Still hard:

- Long-range / semantic edits; rare-word spelling (hallucination).
- Dominant failure = *under-correction*: copying the source unchanged (the safe, high-probability action).

The Final Challenge — Dataset Quality

Why does **every** model score only $\sim$2 % exact match?

Reference quality is the ceiling — not model capacity

1. **References are paraphrases:** *"informed"* → *"suggest"*, *"startups"* → *"start-ups"* — not grammar errors.
2. **References can be noisy:** target *"no brain worky"* — "worky" is not a word.
3. **Many valid corrections exist;** EM credits exactly one.

→ Report GLEU / $F_{0.5}$, not EM. Cleaner data would help more than a bigger model.

Conclusion

- **BoW** detects grammaticality at $\sim$62 % — cannot correct.
- **LSTM** corrects but is limited by the bottleneck (GLEU 0.213).
- **Bahdanau attention** is the decisive jump (GLEU 0.599).
- **Transformer** ties on GLEU (0.618) with 3.6$\times$ fewer params and best validation GLEU (0.613).

Key insight

The binding constraint is **dataset quality**, not architecture — synthetic C4_200M references are often paraphrases or noise.

Thank you

Questions?